

ЛАБОРАТОРНАЯ РАБОТА № 4

Разработка программ с использованием пользовательских функций

Цель: изучение особенностей организации подпрограмм на языке C++; изучение принципов работы с памятью на языке C++; получение навыков разработки программ с использованием пользовательских функций с различными типами аргументов.

Контрольные вопросы для самоподготовки

1. Общие сведения о структуризации программы. Подпрограммы.
2. Особенности пользовательских функций на C++: объявление функции, прототип функции, тип функции.
3. Типы пользовательских функций в C++.
4. Способы передачи аргументов в функцию. Типы и количество параметров функции. Последовательность передачи параметров в функцию.
5. Аргументы функции по умолчанию.
6. Функции с переменным числом параметров.
7. Спецификация inline-функции.
8. Вызов функции и механизм возвращения значений. Стек вызовов.
9. Понятия «область видимости объектов» и «пространство имени переменной» в C++. Соккрытие имен переменных.
10. Глобальная область видимости. Особенности и проблемы при использовании глобальных переменных.
11. Классы памяти при использовании переменных в C++: область действия, время существования, место хранения переменной.
12. Расширение области видимости переменных.
13. Понятие символьной строки. Особенности явного описания символьной строки как массива. Сравнение описаний *char* s* и *char s[]*.
14. Функции работы со символьными строками, описание их аргументов и возвращаемого значения.
15. Аргументы функции *main*.

Задание

1 Согласовать вариант задания с преподавателем, внимательно изучить работу заданной по варианту стандартной функции по работе со строками (таблица 4.1) и пример.

2 Написать программный код реализации следующей задачи :

Определить набор следующих пользовательских подпрограмм:

- функция ввода исходных данных;
- функция вывода результата;
- функция-оболочка для указанной по варианту стандартной функции;
- функция, которая полностью имитирует действия, указанной по заданию стандартной функции.

Реализовать функцию main, которая будет содержать вызовы всех указанных подпрограмм.

Дополнительные требования:

Обращение к элементам строки в пользовательской функции-имитаторе необходимо реализовать через указатели.

Тип возвращаемого значения, типы аргументов и способ их передачи должны полностью совпадать с соответствующими параметрами стандартной функции, действие которой имитируется.

3 Ввод данных пользователем реализовать с клавиатуры, а вывод результатов выполнения программы в консоль (на экран).

4 Проверить правильность вычислений на тестовых примерах, выполнив серию контрольных прогонов программы.

ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ

Вариант	Задание (прототип и описание функции)
0	<code>char*strupr (char* srcptr);</code>
	Функция преобразует буквы нижнего регистра в строке <code>srcptr</code> в буквы верхнего регистра
1	<code>char*strcpy (char* destptr, const char* srcptr);</code>
	Функция копирует строку <code>srcptr</code> , включая завершающий нулевой символ в строку назначения, на которую ссылается указатель <code>destptr</code>
2	<code>char*strncpy (char* destptr, const char* srcptr, size_t num);</code>
	Функция копирует первые <code>num</code> символов из строки <code>srcptr</code> в строку <code>destptr</code> . Если конец строки <code>srcptr</code> достигнут прежде, чем были скопированы все <code>num</code> символов, то к скопированным символам в конец строки <code>destptr</code> добавляется нуль-символ, и строка считается скопированной
3	<code>size_t strlen (const char* str);</code>
	Функция вычисляет длину строки <code>str</code> . Внимание! Не путайте размер массива, который содержит строку, и длину строки
4	<code>int strncmp (const char* str1, const char* str2, size_t num);</code>
	Функция сравнивает первые <code>num</code> символов строки <code>str1</code> с первыми <code>num</code> символами строки <code>str2</code> . Значение большее нуля указывает, что строка <code>str1</code> больше строки <code>str2</code> , отрицательное значение – <code>str1</code> меньше <code>str2</code> . Нулевое значение означает, что строки равны
5	<code>int strcmp (const char* str1, const char* str2);</code>
	Функция сравнивает символы двух строк, <code>str1</code> и <code>str2</code> . Если результат положительный, то строка <code>str1</code> больше строки <code>str2</code> , если результат отрицательный, то <code>str1</code> меньше <code>str2</code> . Нулевой результат, если обе строки равны
6	<code>char*strncat (char* destptr, char* srcptr, size_t num);</code>
	Функция добавляет первые <code>num</code> символов строки <code>srcptr</code> к концу строки <code>destptr</code> . Если строка <code>srcptr</code> больше, чем число <code>num</code> , то после скопированных символов неявно добавляется символ конца строки
7	<code>char*strcat (char* destptr, const char* srcptr);</code>
	Функция добавляет копию строки <code>srcptr</code> в конец строки <code>destptr</code>
8	<code>char*strchr (const char* strptr, int symbol);</code>
	Функция возвращает указатель на последнее вхождение символа <code>symbol</code> в строке <code>strptr</code> .

9	<code>const char* strchr (const char* str, int symbol);</code>
	Функция выполняет поиск первого вхождения символа <code>symbol</code> в строку <code>str</code> . Если значение не найдено, то возвращается нулевой указатель
10	<code>size_t strcspn (const char* str1, const char* str2);</code>
	Функция выполняет поиск первого вхождения в строку <code>str1</code> любого из символов строки <code>str2</code> , и возвращает количество символов до найденного первого вхождения.
11	<code>size_t strspn (const char* str1, const char* str2);</code>
	Функция выполняет поиск символов строки <code>str2</code> в строке <code>str1</code> . Возвращает длину начального участка строки <code>str1</code> которая состоит только из символов, которые являются частью строки <code>str2</code> .
12	<code>const char* strstr (const char* str1, const char* str2);</code>
	Функция ищет первое вхождение подстроки <code>str2</code> в строке <code>str1</code> .
13	<code>char* strtok (char* strptr, const char* delim);</code>
	Функция выполняет поиск лексем в строке <code>strptr</code> . Последовательность вызовов этой функции разбивают строку <code>strptr</code> на лексемы, которые представляют собой последовательности символов, разделенных символами разделителями. Символами разделителями являются символы в строке <code>delim</code> . Если нет найденных лексем, то возвращается пустой указатель. При последующих вызовах функция ожидает в качестве первого аргумента нулевого указателя и использует позицию, следующую за последней лексемой, как начало для сканирования.
14	<code>const void* memchr (const void* memptr, int val, size_t num);</code>
	Функция ищет первое вхождение <code>val</code> (не подписанного символа) в <code>num</code> байтах блока памяти, адресуемого указателем <code>memptr</code> , и возвращает указатель на найденный символ. Если значение не найдено, то возвращается нулевой указатель <code>NULL</code> .
15	<code>const char* strpbrk (const char* str1, const char* str2);</code>
	Функция выполняет поиск первого вхождения в строку <code>str1</code> любого из символов строки <code>str2</code> . Возвращает указатель на первое вхождение найденного символа в <code>str1</code> , либо – пустой указатель, если нет ни одного совпадения.
16	<code>void* memset (void* memptr, int val, size_t num);</code>
	Функция заполняет <code>num</code> байтов блока памяти, через указатель <code>memptr</code> . Код заполняемого символа передаётся в функцию через параметр <code>val</code> .
17	<code>char* _strdup (const char* strcpsrc);</code>
	Функция вызывает <code>malloc</code> для выделения памяти, необходимой для копии <code>strcpsrc</code> , а затем копирует <code>strcpsrc</code> в выделенное пространство. Возвращает указатель на

	расположение хранилища для скопированной строки или <i>NULL</i> , если хранилище не может быть выделено.
18	<code>char* _strlwr (char* strcptr);</code>
	Функция преобразует все буквы в верхнем регистре в строке <code>strcptr</code> в нижний регистр.
19	<code>char* _strrev (char* strcptr);</code>
	Функция изменяет порядок символов в <code>strcptr</code> на обратный.
20	<code>int _sprintf (const char* format [, argument] ...);</code>
	Функция возвращает число символов, которые были бы созданы, если строка была бы напечатана либо отправлена в файл с помощью указанных кодов форматирования. Каждый <code>argument</code> (при наличии) преобразуется согласно соответствующей спецификации формата в параметре <code>format</code>. Формат состоит из обычных символов и имеет те же форму и функциональные возможности, что и аргумент <code>format</code> для функции <code>printf</code>.